



Laboratório 03: Criação e Sincronização de Processos

Chamadas `fork()`, `wait()` e `exec()`

Objetivo

Neste laboratório exploraremos as chamadas de sistema POSIX `fork()`, `wait()` e `exec()` para criação, sincronização e substituição de processos em sistemas operacionais Unix/Linux. Serão realizados experimentos práticos que envolvem a criação de processos filhos, sincronização de execução entre processos e carregamento de programas externos.

Chamada `fork()`

A função `fork()` cria um novo processo duplicando o processo que realiza a chamada. O processo criado (filho) é idêntico ao pai no momento da criação, mas recebe um PID diferente. O valor de retorno da `fork()` permite distinguir os dois processos:

- -1: erro na criação do processo.
- 0: código executado pelo processo filho.
- >0: código executado pelo processo pai.

Chamada `wait()`

A função `wait()` suspende a execução do processo pai até que um de seus filhos finalize. Essa chamada também libera os recursos do processo finalizado e permite obter seu código de saída.

Chamada `exec()`

As funções `exec()` substituem a imagem de um processo pelo conteúdo de outro programa. A função mais comum é `execvp()`, que recebe o caminho do programa e uma lista de argumentos.

Exemplo 1: `fork()` e `wait()`

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/wait.h>
6
7 int main() {
8     int pid, status;
9     pid = fork();
10    if (pid == -1) {
```

```

11     perror("fork falhou!");
12     exit(-1);
13 } else if (pid == 0) {
14     printf("processo filho\t pid: %d\t pid pai: %d\n", getpid(), getppid());
15     exit(0);
16 } else {
17     wait(&status);
18     printf("processo pai\t pid: %d\t pid pai: %d\n", getpid(), getppid());
19     exit(0);
20 }
21 }

```

Exemplo 2: Compartilhamento de dados entre processos

```

1 int main() {
2     int pid, status, k = 0;
3     printf("processo %d\t antes do fork\n", getpid());
4     pid = fork();
5     printf("processo %d\t depois do fork\n", getpid());
6     if (pid == -1) {
7         perror("fork falhou!");
8         exit(-1);
9     } else if (pid == 0) {
10        k += 1000;
11        printf("processo filho\t pid: %d\t K: %d\n", getpid(), k);
12        exit(0);
13    } else {
14        wait(&status);
15        k += 10;
16        printf("processo pai\t pid: %d\t K: %d\n", getpid(), k);
17        exit(0);
18    }
19 }

```

Exercício 1: Árvore de Processos

Implemente um programa que crie uma árvore de 3 processos com chamadas encadeadas de `fork()`. Cada processo deve imprimir "Eu sou o processo XXX, filho de YYY" usando `getpid()` e `getppid()`, e os `wait()` devem garantir a ordem: C antes de B, B antes de A. Use `sleep(1)` entre as mensagens.

Exercício 2: Status de Término

Investigue `WIFEXITED(status)` e `WEXITSTATUS(status)` e modifique o exercício anterior para calcular 5!, com cada processo realizando uma multiplicação e retornando o resultado parcial.

Exercício 3: Terminal Embrionário

Implemente um terminal que leia caminhos completos de programas e execute-os usando `fork()` e `execve()`. Utilize `wait()` ou não dependendo se o comando termina com `&`. O laço deve continuar até que o usuário digite `sair`.